

Realistic Mathematical Analysis of Modern Algorithms which Use SIMD Instructions for Better Performance

Supervisors: Cyril NICAUD
Pablo ROTONDO

Applicant: CHAU Dang Minh

MSc in Mathematics & Computer Science

Gustave Eiffel Univ., in progress

- Relevant coursework: combinatorics, discrete optimization and complexity theory.

MSc in Applied Mathematics

Univ. of Limoges, 2025

- Relevant coursework: stochastic optimization and game theory.

BEng in Computer Science

Univ. of Technology, Vietnam, 2024

- Relevant coursework: data structures and algorithms, and compilers.

Compatibility between Open Automata and Session Types

*M2 internship,
Telecom Paris, 2025*

- **Core Result:** Proved that a simplified OA model is equivalent to multiparty session types, mapping safe composition to subtyping.
- **Takeaway for PhD:** Foundational experience in models of concurrent computing, useful for building models of SIMD parallelism.

Complexity Analysis of the Regex Matcher Hyperscan

*M2 Internship,
LIGM, in progress*

- **Core Analyses:** FDR string matcher (SIMD extension of Shift-Or), correctness of decomposition-based matching, and longest literal in average.
- **Takeaway for PhD:** Techniques for complexity analysis (from analytic combinatorics and probability theory), and experience with SIMD algorithms and their implementation.

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

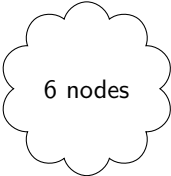
²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.



6 nodes

We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

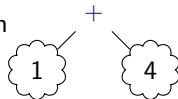
²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

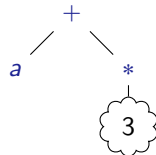
²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

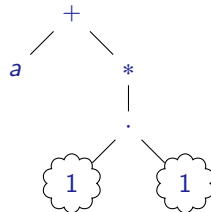
²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

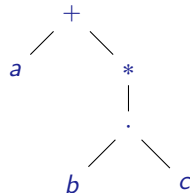
²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

²Binary Search Tree

Research Experience: Algorithms - Details and Progress

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

- 1 matches all literals using FDR (or another string matcher ¹) *at once*,
- 2 verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like ².

Useful statistics to analyze asymptotically, e.g., the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

¹Qiu et al. (2021). Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection.

²Binary Search Tree

Research Experience: Algorithms - Details and Progress

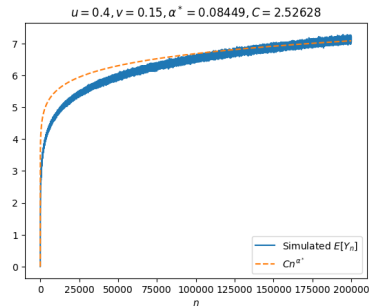
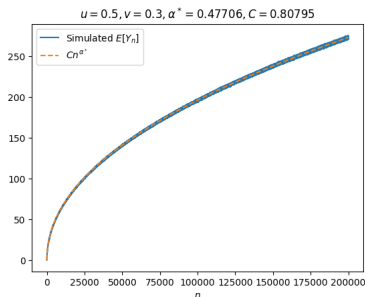
$$Y_{n+2} = \underbrace{I^{(n)}}_{\text{indicator. of root being concat.}} \underbrace{(Y_{U_n} + Y_{n+1-U_n} + 1)}_{\text{take all of the subtrees, plus the current}} + \underbrace{J^{(n)}}_{\text{indicator. of root being alternation}} \underbrace{\max(Y_{U_n}, Y_{n+1-U_n})}_{\text{take the longer of the subtrees}}, \quad Y_1 = Y_2 = 0.$$

For $\mathbb{E}[Y_n]$ when $\mathbb{E}[I^{(n)}] + \mathbb{E}[J^{(n)}] := u + v > 1/2$ (agreeing with real data),

Lower bound $\Omega(n^\alpha)$ where $\alpha > 2u + v - 1$ solves $2(u + v) - v2^{-\alpha} = \alpha + 1$.

Upper bound $O(n^{2(u+v)-1})$

Conjecture: in some domain of (u, v) , $\mathbb{E}[Y_n] \sim Cn^\beta$.



Context and Motivation:

- Developers optimize their library algorithms in a fine-grained way e.g., they make assumptions on the input or leverage hardware features such as SIMD instructions.
- Researchers are interested in explaining if such optimizations are indeed efficient. Examples include both confirmation (TimSort ³) and rejection (Lua's hybrid table ⁴)

Goal: Develop a framework for analyzing algorithms with SIMD parallelism.

Fruitfulness:

- Direct applications to algorithm design.
- Requirements naturally leads to deep mathematical questions, e.g., $\mathbb{E}[Y_n]$ may not be analyzable by classical techniques.

³Auger, N., Jugé, V., Nicaud, C., & Pivoteau, C. (2018). On the worst-case complexity of TimSort.

⁴Martínez, C., Nicaud, C., & Rotondo, P. (2022, October). A probabilistic model revealing shortcomings in Lua's hybrid tables.

Thank you for attention !